

Notes de lecture de Homotopy Type Theory

Eliott Paquet

27 juin 2026

Chapitre 1

Foundation

1.1 Type Theory

1.1.1 Type theory versus set theory

Alternative à ZF, Set theory à deux couches : logique du premier ordre et les systèmes d'axiomes, permet d'analyser les interactions entre set. Contrairement Type theory est son propre système déductif $\longrightarrow$ pas besoin de superstructure. Une notion basique *types*.

Les preuves sont confondues avec construction d'objet (facilement traduit en info?). Dans Set theory une proposition peut être démontré ou réfuter, dans type theory, la preuve est l'appartenance à un type, et ça n'as pas de sens de considérer cette proposition comme fausse ou pas, car $a : A$ est atomique.

Différence dans l'égalité : Set theory $\longrightarrow$ relation entre deux objets (vrai ou pas) type theory : $\longrightarrow$ deux types d'égalité, l'égalité propositionnelle si $a, b : A$ alors, il existe un type $a =_A b$ et l'égalité de définition noté $a \equiv b$

1.1.2 Function Types

Utilisation du lambda calcul dans la représentation des fonctions $(\lambda (x : A) . \varphi) : A \longrightarrow B$ Curryfication des fonctions (prénom de Curry : Haskell!)

1.1.3 Universes and families

Univers est un type où ces éléments sont des types. Pas d'univers de tous les types à cause du paradoxe de Russell. On peut créer une hiérarchie d'univers $\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_3 : \dots$. Pour modéliser une collection de type variant sur un type A on utilise la fonction $B : A \longrightarrow \mathcal{U}$ ces fonctions sont appelées famille de type.

1.1.4 Dependent function type (Π -types)

Les éléments d'un Π -types sont des fonctions dont le type codomaine peut varier en fonction de l'élément du domaine duquel la fonction est appliqué. Exemple : Si $A : \mathcal{U}$ et une famille $B : A \longrightarrow \mathcal{U}$

on peut construire une fonction type-dépendant $\prod_{(x:A)} B(x) : \mathcal{U}$

Principe d'unicité : $f \equiv (\lambda x. f(x))$ pour tout $f : \prod_{x:A} B(x)$.

On peut définir des fonctions polymorphiques comme l'identité telle que $id \equiv \lambda (A : \mathcal{U}) . \lambda (x : A) . x$ ou encore swap :

$$\text{swap} : \prod_{A:\mathcal{U}} \prod_{B:\mathcal{U}} \prod_{C:\mathcal{U}} (A \longrightarrow B \longrightarrow C) \longrightarrow (B \longrightarrow A \longrightarrow C)$$

définie par :

$$\text{swap}(A, B, C, g) := \lambda b. \lambda a. g(a)(b)$$

1.1.5 Product Types

En set theory, quand on définit une fonction sur $A \times B$, on suppose que les couples sont bien ordonnés, ou en type theory, on suppose que la fonction est bien définie, ce qui permet de démontrer que les couples sont des paires.

Pour définir les projecteurs, on peut passer par le récursur pour les types produits défini par le type

$$\text{rec}_{A \times B} : \prod_{C:\mathcal{U}} (A \longrightarrow B \longrightarrow C) \longrightarrow A \times B \longrightarrow C$$

avec la définition

$$\text{rec}_{A \times B}(C, g, (a, b)) \equiv g(a)(b)$$

ce qui permet de dire

$$pr_1 \equiv \text{rec}_{A \times B}(A, \lambda a. \lambda b. a)$$

$$pr_2 \equiv \text{rec}_{A \times B}(B, \lambda a. \lambda b. b)$$

On peut définir le récursur à partir des projections :

$$\text{rec}_{A \times B}(C, g, d) \equiv g(pr_1(d))(pr_2(d))$$

Et il est évident que cette définition du récursur est bien dérivé de cette définition.

Si on a

$$g : \prod_{x:A} \prod_{y:B} C((x, y))$$

alors on peut définir $f : \prod_{x:A \times B} C(x)$ par

$$f((x, y)) \equiv g(x)(y)$$

Ce qui est utile pour prouver le principe d'unicité propositionnelle qui montre que tout élément de $A \times B$ est égal à une paire. Via

$$\text{uniq}_{A \times B} : \prod_{x:A \times B} ((pr_1(x), pr_2(x)) =_{A \times B} x)$$

Lorsque le type sera introduit sous-section 12 on pourra utiliser l'élément réflexif $\text{refl}_x : x =_A x$ pour tout $x : A$ pour définir

$$\text{uniq}_{A \times B} \equiv \text{refl}(a, b)$$

On peut aussi créer la fonction induction pour les types produits :

$$\text{ind}_{A \times B} : \prod_{C:A \times B \longrightarrow \mathcal{U}} \left(\prod_{(x:A)} \prod_{(y:B)} C((x, y)) \right) \longrightarrow \prod_{x:A \times B} C(x)$$

$$\text{ind}_{A \times B}(C, g, (a, b)) \equiv g(a)(b)$$

1.1.6 Dependent pair types